Coverage-Guided Fuzzing of NVIDIA Dynamo: Finding Bugs in a Production LLM Inference Framework

Shounak Ray, Arpit Ranasaria

March 2026

Abstract

We applied coverage-guided fuzzing to NVIDIA Dynamo, an open-source LLM inference framework with over 6,000 GitHub stars. Across 46 fuzz targets spanning 5 Rust crates, we found 16 previously unknown bugs: a critical integer overflow in a network-facing codec, 6 high-severity bugs (panics, deadlocks, logic errors, and data corruption), and 9 medium-severity issues. We submitted a fix for the critical overflow that was approved by NVIDIA's team, and a second bug was independently fixed upstream during our campaign. The most useful thing we learned: the choice of oracle strategy matters more than the fuzzer itself. Crash-only testing missed entire categories of bugs that differential and semantic oracles caught in seconds.

Contents

1 System Description **2**

2 Technical Overview **3**
2.1 Oracle Strategies . . . 3
2.2 Design Evolution . . . 4
2.3 Input Generation and Infrastructure . . . 5

3 System Setup Steps **5**

4 Evaluation Methodology **5**

5 Basic Evaluation **6**
5.1 Bug Discovery Results . . . 6
5.2 Oracle Strategy Effectiveness . . . 7
5.3 Coverage Baseline . . . 7
5.4 Ablation Studies . . . 7
5.5 Discussion . . . 10

6 Evaluation on Production Application **10**
6.1 Notable Bugs . . . 11
6.2 Coverage and Scope . . . 11
6.3 Limitations . . . 11
6.4 Upstream Impact . . . 11

7 Use of AI **12**

8 Software Reproducibility Artifact Package **12**

1 System Description

We used **coverage-guided fuzzing** to find bugs in NVIDIA Dynamo [1], a production LLM inference framework written primarily in Rust. The fuzzer generates random inputs, feeds them to a program, and uses code coverage feedback to steer generation toward unexplored paths. When the program crashes or violates an expected property, the fuzzer saves the triggering input. We use `libFuzzer` [2] via the `cargo-fuzz` [3] Rust wrapper.

Dynamo handles KV-cache-aware request routing, tool-call parsing for function calling, network communication between distributed workers, and token block management. The codebase is roughly 150,000 lines of Rust, actively developed by 200+ contributors, and deployed in production at NVIDIA for LLM serving. Figure 1 shows the architecture and request flow.

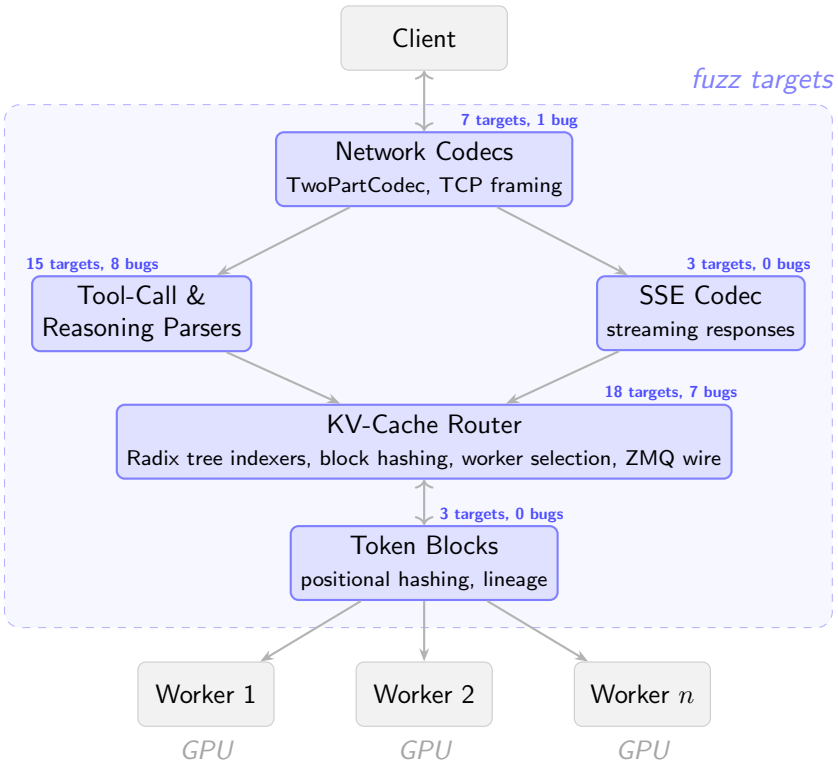

Figure 1: Dynamo architecture with fuzzed components highlighted. Shaded boxes are the 5 crates we targeted. We focused most of our effort on the Parsers and KV-Cache Router, but fuzzed targets lightly across all highlighted components. Arrows show request/response flow from client through network codecs, parsing and routing layers, down to GPU workers.

We targeted 5 core library crates that process untrusted or semi-structured input:

- **lib/parsers**: Tool-call parsers for 14+ LLM output formats (XML, JSON, DSML, Pythonic, GLM-4.7, KimiK2, etc.) plus reasoning parsers (DeepSeek, Granite, MiniMax, etc.)
- **lib/kv-router**: KV-cache routing logic including radix tree indexers, block hash computation, worker selection, and ZMQ wire protocol handling
- **lib/runtime**: Network codecs (TwoPartCodec, TCP framing), event plane serialization, and slug generation
- **lib/tokens**: Token block sequences, positional hashing, and lineage hash computation
- **lib/llm**: Server-Sent Events (SSE) codec for streaming LLM responses

Across these crates, we targeted five classes of bugs: panics and crashes (division by zero, out-of-bounds access, integer overflow, UTF-8 boundary violations), logic errors (incorrect indexer scores, wrong function name extraction, stale cached state), data corruption and loss (silent parameter drops, string value corruption, streaming content loss), concurrency bugs (deadlocks in concurrent data structures), and serialization bugs (encode-decode asymmetries in network protocols).

We chose these crates through a manual code audit. Before writing any fuzz targets, we searched the codebase for dangerous patterns: unchecked indexing, unchecked arithmetic, unsafe string slicing at byte offsets, panicking `unwrap()` calls on external input, and global mutable state. The audit turned up several likely bugs early on, which confirmed these crates were worth targeting and guided our prioritization. We started with the parsers crate (pure-function string processing, easiest to harness) and progressed to the kv-router crate (stateful harnesses, more complex input generation).

2 Technical Overview

This section covers the oracle strategies that define what counts as a bug, how our approach changed based on what we saw working (and not working), and the input generation techniques we used to reach deeper code paths.

2.1 Oracle Strategies

Each of the 5 target crates has its own `fuzz/` directory containing fuzz target source files, seed corpora, dictionaries, and shared helper libraries. The basic structure of a fuzz target is shown below:

Listing 1: Basic crash oracle fuzz target

```
#![no_main]
use libfuzzer_sys::fuzz_target;

fuzz_target!(|data: &[u8]| {
    if let Ok(s) = std::str::from_utf8(data) {
        let _ = some_parser::parse(s);
    }
});
```

Most of our targets go beyond simple crash oracles. In practice, what you define as a bug matters at least as much as which fuzzer you run. We implemented six oracle strategies, each defining "incorrect behavior" differently. Most targets combine multiple strategies (e.g., a differential target also acts as a crash oracle), so the counts below reflect all targets using each strategy, with overlap.

Crash oracles (30+ targets) are the simplest: feed arbitrary bytes to a function, see if it crashes. One useful property of our setup: `cargo-fuzz` compiles targets in debug mode, where Rust enables arithmetic overflow checks, bounds checks, and debug assertions that are stripped in release builds. Integer overflow that silently wraps in production triggers a panic under fuzzing instead. So crash oracles actually catch bugs that would show up as silent data corruption in deployed code. Nearly all of our 46 targets include implicit crash detection, and about 30 use it as their primary oracle. They work best for network codecs and hash functions, where inputs are untrusted and the code was not written to handle arbitrary bytes.

Differential oracles (4 targets) compare two implementations that should produce identical results. We compare streaming vs. one-shot parsing of the same input (any mismatch indicates a bug in one code path), and we run the same sequence of operations against three independent

indexer implementations (RadixTree, ConcurrentRadixTree, PositionalIndexer) and assert their scores match. This "triple differential" approach found bugs that no crash oracle could detect.

Roundtrip oracles (7 targets) serialize data, deserialize it, re-serialize, and assert the bytes are identical. We applied this to the ZMQ wire protocol, the TwoPartCodec network framing, the event plane serialization, and positional hash encodings.

Consistency oracles (10+ targets) define semantic properties that must hold for any valid input: any function name a parser extracts must appear as a substring of the original input (no fabrication), reasoning text must grow monotonically across streaming chunks, radix tree query scores must be bounded by the length of the stored sequence, and hash computations must be deterministic. In the ablation study (Section 5), targets where consistency checking is the primary strategy are labeled "consistency" to distinguish them from crash-only targets.

Semantic oracles (1 target) embed known-valid tool calls (with known function names and argument values) into fuzz-controlled surrounding text, then verify that the parser extracts them correctly. This catches bugs where a parser extracts wrong data rather than crashing.

Stateful/concurrency oracles (5+ targets) generate sequences of operations (store, remove, clear, query) applied to data structures, checking invariants after each step. One target also spawns concurrent threads to test for deadlocks.

2.2 Design Evolution

We did not design this strategy upfront. It evolved as we watched what worked and what didn't.

We started with crash-only harnesses. For the codec and kv-router crates, this worked immediately: the TwoPartCodec crash target found a critical integer overflow within seconds. But the parser crash targets found **zero bugs despite millions of executions**. The parsers are designed to handle arbitrary model output gracefully: malformed input produces an empty result, never a panic.

That null result forced us to rethink. We switched to differential and semantic oracles that define what "correct" means rather than waiting for the program to crash. The differential harness quickly found parser bugs that crash-only testing had missed entirely. Same code, same fuzzer, different oracle, different results. The quantitative comparison is in Section 5.

The same pivot worked for kv-router. We built a stateful harness that generates sequences of store, remove, clear, and query operations against the radix tree, checking invariants after each step. This found an aliasing panic with a 10-byte input. We then built a differential harness comparing the single-threaded `RadixTree` against the `ConcurrentRadixTree` and `PositionalIndexer`, which uncovered a deadlock in the concurrent implementation (Bug 2) and a score underreporting bug (Bug 15) that NVIDIA's team later fixed independently.

We then ran LLVM coverage reports over the accumulated corpus to find blind spots. Several parsers had 0% coverage from generic targets because they required format-specific input structure. We built targeted harnesses for those (e.g., `fuzz_glm47_utf8` for multibyte handling, `fuzz_xml_deep` with structured inputs for nested argument extraction), which turned up additional bugs.

Finally, we had to deal with known bugs dominating fuzzing results. One crash was so trivially triggerable that it caused the fuzzer to crash and restart on nearly every input, blocking exploration of deeper code paths. The standard workaround of catching panics in the harness was not available: AddressSanitizer, which `cargo-fuzz` enables by default, intercepts panic unwinding and aborts the process. We solved this by detecting and skipping inputs that trigger known bugs at the harness level, checking preconditions before passing input to the function under test. With this filtering, the same targets found additional bugs that had been unreachable before.

2.3 Input Generation and Infrastructure

Many targets use the `Arbitrary` trait [4] to generate structured fuzz inputs rather than raw bytes. This way, mutations are more likely to produce valid-looking inputs that reach deeper code paths instead of being rejected by format validation. Section 5 quantifies the coverage advantage.

Each fuzz crate includes seed corpora (hand-crafted valid inputs for each format), token dictionaries (format-specific keywords), and shared helper libraries for common operations like chunking strings along UTF-8 boundaries and constructing valid event structures. The full source for all targets and infrastructure is available in our repository.[1]

3 System Setup Steps

All code, fuzz targets, seed corpora, dictionaries, crash artifacts, and scripts are in the `fuzzy-dynamo` repository, a fork of `ai-dynamo/dynamo`. The following steps reproduce our setup:

1. **Clone the repository:** `git clone https://github.com/ShounakRay/fuzzy-dynamo.git && cd fuzzy-dynamo`
2. **Install system libraries.** On Ubuntu: `sudo apt install build-essential libhwloc-dev libudev-dev pkg-config libclang-dev protobuf-compiler python3-dev cmake`. On macOS: `brew install cmake protobuf`.
3. **Install Rust:** `curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh` and then install the nightly toolchain and cargo-fuzz: `rustup install nightly && cargo +nightly install cargo-fuzz`.
4. **Run all fuzz targets:** `./fuzzing/run.sh` (defaults to 60s per target). Filter by crate (`--crate parsers`) or target (`--target fuzz_differential`). Adjust timeout with `FUZZ_TIMEOUT=300`.
5. **Run individual targets directly:** `cd lib/parsers && cargo +nightly fuzz run fuzz_tool_call_par -- -max_total_time=60`.
6. **Generate LLVM coverage reports:** `./fuzzing/coverage.sh`.

Full build-from-source instructions (including Python bindings and optional GPU components) are in the repository README.

4 Evaluation Methodology

We evaluate along three dimensions: what bugs were found and how, how much code was exercised, and how oracle strategy choice affects outcomes.

Metrics. For each fuzz target we record coverage edges (unique control-flow transitions exercised), coverage features (a finer-grained metric including edge hit-count buckets), corpus size (unique coverage-contributing inputs), executions per second, total executions, new crashes, and peak memory usage.

Bug detection. Bugs are detected through four mechanisms: natural panics (Rust panics on out-of-bounds access, integer overflow, unwrap failures), assertion oracles (explicit assertions in fuzz targets that flag undesirable non-crashing behavior), overflow checks (optionally enabled to catch integer overflows that silently wrap in release mode), and AddressSanitizer (integrated by default for memory safety issues).

Oracle validation. We validate each oracle type by confirming it can detect known bugs. For each oracle strategy, we constructed or identified a known-buggy input and verified that the harness flags it within seconds. For example, we verify that the crash oracle rediscovers known panics, that

[1] `https://github.com/ShounakRay/fuzzy-dynamo`

the differential oracle catches a known streaming mismatch, and that the roundtrip oracle catches the TwoPartCodec integer overflow using a saved crash artifact.

Ablation study. To quantify the impact of oracle strategy choice, we designed two types of comparisons. First, our `ablation.sh` script runs all 46 targets at three time budgets (10s, 30s, 60s), recording coverage, corpus size, throughput, crashes, and time-to-first-crash for each. This provides aggregate data across oracle types and time budgets, though different oracle types are applied to different code, which limits direct comparison.

Second, to isolate individual variables, we built **controlled pair experiments**: pairs of fuzz targets that test the same underlying code with exactly one variable changed. For input structure, we created `Arbitrary`-derived structured variants alongside existing raw-byte targets for the same functions (e.g., `fuzz_zmq_wire_roundtrip` vs. `fuzz_zmq_wire_roundtrip_structured`). For known-bug filtering, we created unfiltered variants that omit the guards skipping known crashes (e.g., `fuzz_request_extra_info` vs. `fuzz_request_extra_info_unfiltered`). Each pair was run with an empty corpus at 10s, 30s, and 60s to ensure fair comparison.

Production application. The system under test is NVIDIA Dynamo itself: actively maintained by 200+ contributors, with an existing test suite, deployed in production for LLM serving. All bugs we report were found in this production codebase, not in hand-crafted test programs. Section 6 evaluates the real-world impact.

5 Basic Evaluation

This section reports quantitative results: the bugs found, coverage achieved, and ablation studies comparing oracle strategies, time budgets, and input generation methods.

5.1 Bug Discovery Results

Table 1 summarizes all 16 bugs. Each was found by the fuzzer through crash artifacts or oracle violations. The code audit described in Section 1 guided which components we targeted and how we designed the harnesses.

Table 1: Summary of all bugs discovered

#	Component	Bug Type	Severity	Discovery	Status
1	kv-router	`chunks_exact(0)` panic	Medium	Crash oracle	Open
2	kv-router	RwLock deadlock	High	Stateful oracle	Open
3	parsers	UTF-8 boundary panic	High	Crash oracle	Open
4	kv-router	Jump optimization skip	High	Differential oracle	Open
5	kv-router	ZMQ wire OOB panic	High	Crash oracle	Open
6	parsers	Regex prefix absorption	High	Semantic oracle	Open
7	parsers	Silent parameter drop	Medium	Crash oracle	Open
8	parsers	Streaming content loss	Medium	Differential oracle	Open
9	parsers	OnceLock stale cache	Medium	Crash oracle	Open
10	parsers	Trailing newline mismatch	Medium	Differential oracle	Open
11	kv-router	Division by zero	Medium	Crash oracle	Open
12	kv-router	Division by zero	Medium	Crash oracle	Open
13	parsers	`strip_quotes` panic	Medium	Crash oracle	Open
14	parsers	`try_literal_eval` corruption	Medium	Crash oracle	Open
15	kv-router	Score underreporting	High	Differential oracle	Fixed
16	runtime	Integer overflow	Critical	Crash oracle	Fixed

5.2 Oracle Strategy Effectiveness

Different oracle strategies found different classes of bugs (Table 2). Crash oracles found 10 bugs (panics, overflows, out-of-bounds access, data corruption), differential oracles found 4 (streaming/one-shot mismatches, score underreporting, content loss), the semantic oracle found 1 (regex prefix absorption), and the stateful oracle found 1 (deadlock). Crash-only testing was completely blind to logic errors like score underreporting and streaming mismatches. Without the non-crash oracle types, we would have missed 6 of the 16 bugs.

Table 2: Bugs found per oracle strategy

Oracle Strategy	Bugs Found	Examples
Crash-only	10	Division by zero, OOB access, integer overflow, UTF-8 panic, data corruption
Differential	4	Streaming/one-shot mismatch, score underreporting, content loss
Semantic	1	Regex prefix absorption in function name extraction
Stateful	1	Deadlock on duplicate block hashes

5.3 Coverage Baseline

Coverage and throughput vary by orders of magnitude across targets (Table 3). This mostly reflects differences in code complexity, not fuzzing quality. Throughput alone is misleading: the fastest target (684K exec/s) covers only 90 edges, while the slowest (285 exec/s) covers 6,951. Because of this variance, the remaining subsections use controlled comparisons to isolate the variables that actually matter.

Table 3: Coverage by crate at 60 seconds (targets with >0 coverage)

Crate	Representative Target	Oracle	Cov Edges	Features	Corpus	Exec/s
parsers	fuzz_parser_stress	consistency	7,864	30,580	1,723	3,188
parsers	fuzz_differential	differential	6,951	—	308	285
parsers	fuzz_deepseek_variants	crash	5,582	15,526	1,842	7,142
parsers	fuzz_glm47_utf8	crash	3,843	13,686	197	<1
kv-router	fuzz_zmq_wire_decode	crash	3,481	4,827	1,493	12,637
kv-router	fuzz_zmq_wire	differential	3,337	8,328	2,233	13,539
kv-router	fuzz_kv_protocol_json	crash	2,607	9,170	2,590	12,134
kv-router	fuzz_radix_tree_events	stateful	1,030	6,372	738	6,335
runtime	fuzz_network_codecs	crash	2,535	5,348	1,313	13,673
runtime	fuzz_envelope_roundtrip	differential	1,709	3,220	1,034	14,954
tokens	fuzz_token_block_seq	crash	379	2,060	302	61,045
tokens	fuzz_positional_hash	roundtrip	90	97	10	684,167

5.4 Ablation Studies

Time scaling. Table 4 shows how coverage scales with time budget. Complex targets kept growing: `fuzz_parser_stress` gained 16.5% more edges from 10s to 60s, `fuzz_zmq_wire_decode` gained 29.1%. Simple targets saturated almost immediately: `fuzz_radix_tree_consistency` stayed at

exactly 525 edges across all three timeouts. For simple, low-branching code, even 10 seconds is enough. Complex parsers and protocol handlers benefit from longer runs.

Table 4: Coverage and corpus growth across time budgets

		Coverage Edges			Corpus Size		
Target	Oracle	10s	30s	60s	10s	30s	60s
fuzz_parser_stress	consistency	6,748	7,380	7,864	865	1,137	1,723
fuzz_deepseek_variants	crash	5,005	5,432	5,582	878	1,349	1,842
fuzz_zmq_wire_decode	crash	2,697	3,130	3,481	1,099	1,304	1,493
fuzz_kv_protocol_json	crash	2,516	2,561	2,607	2,261	2,417	2,590
fuzz_envelope_roundtrip	differential	1,703	1,705	1,709	992	1,012	1,034
fuzz_sse_decode_data	crash	1,168	1,177	1,177	2,215	2,271	2,338
fuzz_radix_tree_events	stateful	1,030	1,030	1,030	709	719	738
fuzz_radix_tree_consistency	stateful	525	525	525	77	78	82

Oracle strategy comparison. Table 5 aggregates results by primary oracle type at 60 seconds. We include this for completeness, but with a caveat: different oracle types are applied to different code, so differences in crash counts or coverage may reflect the underlying code, not the oracle. The stronger evidence for oracle effectiveness comes from the bug inventory (Table 2), where we can trace each bug to the oracle that found it, and from the controlled comparison below.

Table 5: Aggregate results by oracle strategy at 60 seconds

Oracle Strategy	Targets	Avg Coverage	Total Corpus	Total Crashes	Avg Exec/s
Crash-only	21	1,388	15,272	3	14,494
Differential	8	2,008	4,352	4	27,513
Consistency	9	2,603	4,000	2	4,324
Stateful	5	1,216	4,355	0	7,906
Roundtrip	1	90	10	0	684,167
Property	1	0	0	1	0
Semantic	1	0	0	1	0

The cleanest controlled comparison comes from three targets that all test the same reasoning parser code (`base_parser.rs`): `fuzz_reasoning_oneshot` (crash oracle), `fuzz_reasoning_streaming` (consistency oracle), and `fuzz_differential` (differential oracle). The differential target found its bugs (streaming/one-shot mismatches) in 4–6 seconds. The crash and consistency targets found different bugs (a panic in the oneshot path and a monotonicity violation), but only at the 60-second timeout (89s and 104s). One comparison does not prove a general claim. But the pattern is clear: the differential oracle defines "incorrect" more precisely, so the fuzzer finds violations faster and with smaller inputs.

Time to first crash. Table 6 shows that most crash bugs are shallow: 8 of 11 crash-producing targets found their first crash within 6 seconds. Three targets required sustained exploration (62–104 seconds) and were only discoverable at the 60-second timeout. A quick 10-second sweep catches shallow bugs, but 60 seconds is needed for bugs behind complex state machines.

Table 6: Time to first crash by target (at 60-second timeout)

Target	Oracle	Time to Crash	Also Found at 10s?
fuzz_parser_semantic	semantic	2s	Yes
fuzz_parser_properties	property	2s	Yes
fuzz_tool_call_parsers	crash	2s	Yes
fuzz_glm47_utf8	crash	2s	Yes
fuzz_block_hash	consistency	5s	Yes
fuzz_request_extra_info	differential	5s	Yes
fuzz_triple_differential	differential	4s	Yes
fuzz_differential	differential	6s	Yes
fuzz_two_part_roundtrip	differential	62s	No
fuzz_reasoning_oneshot	crash	89s	No
fuzz_reasoning_streaming	consistency	104s	No

Structured vs. unstructured inputs. To isolate the effect of input structure, we built controlled pairs: a raw-byte target and a structured `Arbitrary`-derived target that call the same functions. Table 7 shows the results. Structured inputs gained between +16% (ZMQ roundtrip) and +668% (block hash computation) more coverage. The reason is simple: raw bytes must pass format validation before reaching the code under test, and most random bytes fail immediately. Structured inputs satisfy the format constraints by construction, so the fuzzer spends its time on the function's actual logic instead of getting rejected at the entrance.

Table 7: Structured vs. unstructured input coverage (controlled pairs, empty initial corpus)

		Coverage Edges			Exec/s		
Target Pair	Input	10s	30s	60s	10s	30s	60s
ZMQ wire roundtrip	raw bytes	560	560	560	9,774	8,053	9,033
	structured	649	650	650	13,008	13,508	13,214
	delta	*+16%*	*+16%*	*+16%*	*+33%*	*+68%*	*+46%*
Block hash	raw bytes	25	25	25	—	—	—
	structured	58	107	192	—	—	—
	delta	*+132%*	*+328%*	*+668%*			

The ZMQ roundtrip pair is the cleaner comparison: neither variant crashes, both run for the full timeout, and the raw-byte version already gets reasonable coverage because msgpack deserialization accepts a wide range of byte sequences. Even so, structured inputs get 16% more coverage edges, 34% more features, and 46% higher throughput. The block hash pair is more extreme: the raw-byte version stagnates at 25 edges because most random 4-byte sequences produce a `kv_block_size` that triggers an early return. The structured version generates valid parameter combinations directly and reaches nearly 8× the coverage at 60s.

Known-bug filtering. We created unfiltered variants of two targets that normally skip inputs triggering known bugs. Table 8 compares the filtered and unfiltered versions of `fuzz_request_extra_info`, where the filtered version guards against `block_size=0` (a known division-by-zero). Both variants crash, but the filtered version finds more new coverage-contributing inputs at sustained timeouts (9 vs. 8 at 60s, 7 vs. 2 at 30s) because it can explore code paths beyond the trivially triggerable crash. The unfiltered version keeps hitting the same division-by-zero, and libFuzzer spends time minimizing and re-discovering that same crash instead of exploring deeper code.

Table 8: Known-bug filtering: request_extra_info (empty initial corpus)

	Coverage Edges			New Coverage Inputs		
Variant	10s	30s	60s	10s	30s	60s
Filtered (skip `block_size=0`)	19	58	60	0	7	9
Unfiltered (no guard)	55	43	54	3	2	8

5.5 Discussion

Three things stand out from the data, each backed by a different type of evidence.

The oracle defines what the fuzzer can find. This is the strongest finding, and it rests on the bug inventory (Table 2), not aggregate performance comparisons. Of the 16 bugs, 6 are invisible to crash oracles: streaming/one-shot mismatches produce silently wrong output, score underreporting returns a valid but incorrect number, content loss drops data without panicking, and the deadlock hangs instead of crashing. These bugs live in Rust code that handles malformed input gracefully. No amount of fuzzer throughput finds a bug that the oracle cannot define as incorrect. The controlled comparison on reasoning parsers (Section 5.4) makes this concrete: the differential oracle found its bugs in 4–6 seconds, while the crash oracle on the same code needed 89 seconds to find a different, shallower bug.

Most crash bugs are shallow, but a longer budget catches a few more. 8 of 11 crash-producing targets found their first crash within 6 seconds (Table 6). A quick sweep catches those. But three targets (`fuzz_two_part_roundtrip`, `fuzz_reasoning_oneshot`, `fuzz_reasoning_streaming`) only found crashes at the 60-second timeout. These involve more complex state: the TwoPartCodec roundtrip requires generating a valid message header before the overflow is reachable, and the reasoning parsers require exploring streaming state machine transitions. The time-scaling data in Table 4 lines up: complex targets like `fuzz_parser_stress` gained 16.5% more coverage from 10s to 60s, while simple targets like `fuzz_radix_tree_consistency` saturated immediately.

Input structure and known-bug filtering both improve coverage. The structured-input experiments (Table 7) show a 16–668% coverage advantage from `Arbitrary`-derived inputs. Raw bytes must survive format validation before reaching the code under test, and survival rates are low for structured formats. The ZMQ roundtrip pair shows a modest 16% gain because msgpack tolerates arbitrary bytes; the block hash pair shows a 668% gain because the function requires a valid `kv_block_size` in the first 4 bytes, which random bytes almost never produce. The filtering experiment (Table 8) shows a similar dynamic: when the fuzzer keeps hitting a known crash, it spends time minimizing and re-discovering that same bug rather than exploring new code. Filtering the known crash lets the fuzzer move past it, finding 3.5× more new coverage-contributing inputs at the 30-second mark.

One thing that does not show up in the quantitative data but mattered in practice: coverage-driven harness refinement. We used LLVM coverage reports to find parsers with 0% coverage from generic targets, then built targeted harnesses for those parsers. This led directly to the UTF-8 boundary panic (Bug 3) and the data corruption bug (Bug 14).

6 Evaluation on Production Application

All 16 bugs were found in production NVIDIA Dynamo code, not in toy programs or hand-crafted benchmarks. Dynamo is actively developed by a large team with an existing test suite. This section

looks at the real-world significance of those bugs, the scope of our testing, and where fuzzing fell short.

6.1 Notable Bugs

Four bugs are worth describing in detail:

TwoPartCodec Integer Overflow (Bug 16, Critical). The `decode()` function in Dynamo's network codec computes a message size using unchecked addition. In release mode, large values silently wrap to a small number, which can bypass the maximum message size check. Any network peer can trigger this with a crafted 24-byte message header. We filed an upstream issue[2] and submitted a fix as PR #6959,[3] which NVIDIA's team reviewed and approved.

ConcurrentRadixTree Deadlock (Bug 2, High). When store events contain duplicate block hashes, the radix tree creates a self-referential node. A write lock is held while a read lock is attempted on the same lock, so the process hangs silently with no error message. Any worker sending KV cache events with hash collisions can trigger this.

GLM-4.7 UTF-8 Boundary Panic (Bug 3, High). The GLM-4.7 parser uses byte offsets from a trimmed string to index into the original untrimmed string. Multibyte UTF-8 characters (common in Chinese model outputs) cause the byte offset to land in the middle of a character, triggering a panic.

XML Parser Data Corruption (Bug 14, Medium). The `try_literal_eval` function uses global string replacement to convert Python literals to JSON (`"True"` → `"true"`), but this corrupts string values containing these keywords as substrings: `"TrueNorth"` becomes `"trueNorth"`, silently corrupting tool call arguments.

6.2 Coverage and Scope

Our 15 parser targets collectively exercise all 14+ parser implementations across both streaming and one-shot paths. The 18 kv-router targets cover the main code paths in radix tree operations, block hash computation, worker selection, and ZMQ wire protocol handling. The 7 runtime targets cover the TwoPartCodec, TCP framing, and event plane serialization.

6.3 Limitations

Functions requiring a Tokio async runtime or network connections could not be fuzzed directly; we worked around this by fuzzing the synchronous codec and parsing layers underneath the async interfaces. Dynamo's backend integrations (TensorRT-LLM, vLLM, SGLang) require NVIDIA GPUs and could not be fuzzed on our macOS development machines. Some code paths are only reachable with specific configurations the fuzzer cannot easily discover, like model architectures that enable certain parser features.

6.4 Upstream Impact

Two bugs were confirmed upstream: we filed an issue and submitted a fix for Bug 16 (Issue #6955, PR #6959), which NVIDIA's team reviewed and approved, and Bug 15 was independently fixed by the Dynamo developers in PRs #5973/#6122 during our campaign. The remaining 14 have proposed fixes in individual issue writeups in our repository, each with reproduction steps, root cause analysis, and suggested code changes.

[2] `https://github.com/ai-dynamo/dynamo/issues/6955`
[3] `https://github.com/ai-dynamo/dynamo/pull/6959`

Beyond bug reports, we contributed a UTF-8 safety fix (a shared `chunk_ends_with_token_prefix()` function replacing unsafe byte-based slicing across 4 parser files) and 90+ regression tests added to the production source code.

7 Use of AI

We used Claude (Anthropic's LLM) as a coding assistant throughout the project.

Codebase exploration and target discovery. Dynamo is ∼150K lines of Rust across dozens of crates, and we had never worked in it before. Claude helped us build context on the repository: understanding crate boundaries, identifying which modules process untrusted input, and narrowing focus to the five crates we ended up targeting. This was much faster than reading the codebase cold.

Harness development. Claude generated boilerplate fuzz harness code and adapted oracle patterns across similar targets. Once we had a working differential oracle for one parser, Claude adapted it to all 14+ parser implementations with the correct type signatures and configuration.

Debugging and infrastructure. Claude diagnosed platform-specific issues (macOS vs. Linux compatibility in shell scripts, libFuzzer dictionary format constraints), analyzed crash artifacts to identify root causes, and helped structure the ablation study scripts.

Documentation. Claude drafted bug writeups with reproduction steps and root cause analysis, which we reviewed and edited.

We reviewed, tested, and edited all AI-generated code and text. The bugs were found by the fuzzer, not the AI. Claude's contribution was speeding up the engineering work around the fuzzing campaign, not the bug discovery itself.

8 Software Reproducibility Artifact Package

Everything is in the `fuzzy-dynamo` repository.[4] It contains all 46+ fuzz targets across 5 crates (including controlled-pair variants for the structured-input and known-bug-filtering experiments), seed corpora and dictionaries for each target, saved crash artifacts for all bugs (replayable with `cargo +nightly fuzz run <target> <artifact>`), the unified runner (`fuzzing/run.sh`), LLVM coverage report generator (`fuzzing/coverage.sh`), 16 individual bug writeups with reproduction steps in `fuzzing/issues/` and `upstream/`, and 90+ regression tests in the production source code. The setup script (`setup.sh`) configures git remotes and hooks for a one-command start.

References

[1] NVIDIA, "Dynamo: A disaggregated LLM inference framework," https://github.com/ai-dynamo/dynamo, 2024–2026.

[2] LLVM Project, "libFuzzer – a library for coverage-guided fuzz testing," https://llvm.org/docs/LibFuzzer.html.

[3] Rust Fuzzing Authority, "cargo-fuzz: Command-line wrapper for libFuzzer," https://github.com/rust-fuzz/cargo-fuzz.

[4] Rust Fuzzing Authority, "arbitrary: Generating structured data from raw bytes," https://github.com/rust-fuzz/arbitrary.

[4] https://github.com/ShounakRay/fuzzy-dynamo